

Vertical and horizontal diffusion algorithm

Felix Segin

See output.txt for the complete list of modules in src. When creating a DiffusionMap, we use the following algorithm.

I. Ensemble

First, we create the ensemble.

In `self.getPrecincts()`, the Ensemble creates one Precinct object for each precinct in `ct.network`. The Precinct class is not mathematically interesting and acts only as a vessel for precinct-specific data:

Precinct ID indexed from 0 to 694, inclusive (int)

Precinct county (str)

Precinct name (str)

A list containing the IDs of neighbouring precincts (list[int])

Precinct population (int)

The number of votes from the precinct for Biden (Democrat) in 2020 (int)

The number of votes from the precinct for Trump (Republican) in 2020 (int)

The precinct's land area (float)

The length of the precinct's border (float)

Note that the sum of votes for Biden and Trump is less than the population because not all precinct residents vote). The Precinct class also has functions for string representation, a dunder equality method (by precinct ID), a hashing method, and a method `self.doesNeighbour()` for determining whether two precincts neighbour each other.

Then, in `self.getDistricts()`, the Ensemble creates District objects from `small.jsonl`. This file contains 97 distinct districting plans, each comprised of 6 districts indexed from 1 to 6. When creating a DiffusionMap, you must specify a parameter `up_to` which controls the number of districting plans that are used. i.e. the number of districts is six times `up_to`.

A District is also not very mathematically interesting, acting as a vessel for district-specific data:

The districting plan the district came from, indexed 0 to 96 (int)

The index of the district within its districting plan, indexed 0 to 5 (int)

The district's unique ID amongst all districts, calculated by `'district + 6*plan'` (int)

A list of the precincts contained in the district as Precinct objects (list[Precinct])

Vertical and horizontal diffusion algorithm

Felix Sestin

District population, computed by summing across member precincts (int)

The number of votes for Biden from the district (Democrat) in 2020, summed (int)

The number of votes for Trump from the district (Republican) in 2020, summed (int)

The precinct's land area, summed (float)

The 'core' of the district.

The core of the district is computed within the class Core as follows, here we suppose that we use population data:

- 1) For a district containing n precincts, we construct the undirected graph Laplacian L of the precinct graph by $L = D - A$, where A is the adjacency matrix of the precinct graph with $A_{ij} = 1$ if and only if precincts i and j border and zero otherwise, and D is the degree matrix.
- 2) Then, we suppose an $(n + 1)$ th vertex of the graph representing the outside. Edges touching this vertex are directed, and a random walker on the graph can diffuse from a precinct/vertex to the outside if and only if the precinct is on the outer boundary of the district. However, that random walker cannot diffuse anywhere from the outside, and additionally we ascribe a degree of 1 to the outside.
- 3) Update the graph Laplacian L with the outside vertex. The actual effect of this is that we add a new entry -1 to $L_{i(n+1)}$ for every i corresponding to an exterior precinct and an entry 1 to $L_{(n+1)(n+1)}$. The bottom row should be all zeroes except for the 1 in the bottom-right corner.
- 4) Then, we construct the forcing vector (column) $f \in \mathbb{R}^{n+1}$. The i th entry of f should be the population of the i th precinct in the district, and the last entry $(n + 1)$ of f is set to zero to enforce the Dirichlet boundary condition.
- 5) Last, we construct and solve the linear system $Lu = f$ for $u \in \mathbb{R}^{n+1}$. The last entry of u is zero; we truncate this last entry to get a vector $\hat{u} \in \mathbb{R}^n$.

This vector $\hat{u}$ is what we refer to as the core of the district, and it should be thought of as a function assigning a weight to each precinct in the district. It is meant to capture two notions: first, some notion of the interior, or geographic “core” of the district; and second, some notion of the population centre, or demographic “core” of the district.

Note that population data is not the only possible type of data we can input into the forcing vector. We may use equal weights (all ones), partisan votes, or any type of data we have stored in Precinct objects. Changing the type of data may lead to interesting patterns.

Vertical and horizontal diffusion algorithm

Felix Sestin

The purpose of the core is twofold. First, it will be normalized (so it sums to one) and we will perform Metropolis-Hastings on it to get the vertical diffusion matrix for each district. Second, it will be used in the distance function that we use to define horizontal diffusion.

The core is computed once per District object, at the time of initialization of the District.

The District class also contains a method for string representation and a dunder equality method based on unique district ID.

Last, in `self.getNodes()`, the Ensemble creates Node objects from the set of Precinct and District objects that it has created. A Node is a district-precinct pair and not very interesting on its own, mathematically. The method iterates through each district and within each district iterates through each precinct, creating a new Node object for each district-precinct pair. Each Node has a unique ID which is just a tuple of the member district's ID and the member precinct's ID.

In addition to a dunder equality method based on node ID, the Node class contains a few interesting and useful methods:

- 1) `self.canVerticallyDiffuseTo(other)` returns True if the nodes can vertically diffuse to each other, that is, if their districts are the same and their precincts neighbour.
- 2) `self.canHorizontallyDiffuseTo(other)` returns True if the nodes can horizontally diffuse to each other, that is, if their districts are distinct and their precincts are the same.

Distance is defined between two nodes and so there are two different distance functions contained within the Node class, which can be specified by the parameter 'method' and further options can be specified by the parameter 'mode'. Note that distance between nodes really depends only on distance between their member districts, so we will speak of the distance between a pair of nodes and the distance between a pair of districts in the same way.

- 1) `method = 'core'`:
 - a. `mode = (k: int, 'normal')`
 - i. First, we fit each core to a normal distribution by computing the mean μ and standard deviation σ of the core's entries and then replacing each entry x with $\frac{x-\mu}{\sigma}$.
 - b. `mode = (k: int,  $\mu$ : float,  $\sigma$ : float)`
 - i. Using the same given numbers μ, σ for both cores, we replace every entry x in either core with $\frac{x-\mu}{\sigma}$. I'm not quite sure why you would want to do this but it's a place to experiment.
 - c. `mode = (k,  $\mu_1, \sigma_1, \mu_2, \sigma_2$ )`

Vertical and horizontal diffusion algorithm

Felix Sesin

- i. Similar to the previous mode, but with more flexibility. The entries x of the first core are replaced with $\frac{x-\mu_1}{\sigma_1}$ and the entries y of the second core are replaced with $\frac{y-\mu_2}{\sigma_2}$. More appealing than the previous mode.
- d. Once we have performed one of the above-described affine transformations on the cores, we compute the core distance. We iterate through every precinct contained in either district and get the value from each district's core corresponding to that precinct. If a precinct is not in a district (so the core has no value for that precinct), then we just assume a value of zero. In this way, if there are N distinct precincts contained in either district, then the cores, which are vectors in $\mathbb{R}^n$ with $n < N$, get augmented with entries of zero to be vectors in $\mathbb{R}^N$. Then, we take a k -norm of the difference between those two vectors, with k coming from the first entry of mode.

$$\text{dist}(d_1, d_2) = \left(\sum_{p \in d_1 \cup d_2} |\text{core}_1(p) - \text{core}_2(p)|^k \right)^{\frac{1}{k}}$$

2) method = 'hamming':

- a. There are several different modes for Hamming, each a string corresponding to some method of weights for precincts like equal weights, population weights, partisan weights, etc. The basic principle comes from the Hamming distance and goes as follows: any precinct in one district but not in the other, i.e. any precinct in the symmetric difference of the districts, is a difference between the districts. The Hamming distance is computed by summing the features of the differing precincts.
- b. Equal weights correspond to the cardinality of the symmetric difference, whereas population weights correspond to the number of people living in one district but not in both.

II. Vertical

Here, the DiffusionMap object calls on the class Vertical to produce the vertical diffusion matrix V representing vertical diffusion between nodes for the set of nodes. V is a row-stochastic matrix. For computation complexity purposes, we use scipy's built-in sparse linear algebra.

Because vertical diffusion occurs only within districts and we have indexed nodes by district and then by precinct, you can imagine that V is block-diagonal, with each block describing the vertical diffusion within a district. Therefore, our approach for construction is to construct the vertical diffusion matrix for each district, package them up into a list, and then use scipy's block-diagonal constructor.

So, we turn our attention back to a single district. We want a random walker on this district to spend more time in the core of the district, so we want to turn the core into a distribution π (so it sums to one) and then use the Metropolis-Hastings algorithm to find a transition matrix T satisfying $\pi T = \pi$.

When we normalize the core, we have the option to transform the data first. We can subtract from every entry of the core some number between 0 and the minimum entry in the core. Call this number s . In the case $s = 0$, we subtract nothing and the core is left intact. In the case $s = \min(\text{core})$, the smallest entry or entries of the core get sent to 0, which emphasizes the precincts weighted greater by the core even more.

After this transformation, we divide the core by the sum of its entries to turn it into a distribution and write it as a row vector. This is the vector π we will use for Metropolis-Hastings.

In Metropolis-Hastings, we also require an initial transition matrix Q . We have two options for Q .

The first option is an equal-probability random walk between neighbouring precincts in the district. That is, suppose the i th precinct has m neighbours. Then, we have $Q_{ij} = \frac{1}{m}$ if precinct j neighbours precinct i and $Q_{ij} = 0$ otherwise. This makes Q a row-stochastic matrix with zeroes on the diagonal, and is the most natural choice for Q .

The second option is an equal-probability random walk between all precincts. This is like the first option, except we assume that each precinct neighbours every other precinct. If there are n precincts, then every off-diagonal entry is $\frac{1}{n-1}$ and every diagonal entry is zero. While it is row-stochastic with zeroes on the diagonal and not biased in any particular direction, it is a much less natural choice for vertical diffusion.

Armed with π and Q , we perform the Metropolis-Hastings algorithm to get a matrix T . Essentially, we calculate $\alpha_{ij} = \min\left\{1, \frac{\pi_j Q_{ji}}{\pi_i Q_{ij}}\right\}$ for every pair $i \neq j$ and get the off-diagonal entries

Vertical and horizontal diffusion algorithm

Felix Segin

of T by $T_{ij} = Q_{ij}\alpha_{ij}$. Then, we compute the diagonal entries of T by filling every (i, i) th entry so that the i th row sums to one.

This gives us a matrix T with $\pi T = \pi$, and this will be our transition matrix for vertical diffusion on the district. We compute T for every district and then make these the blocks for our block-diagonal matrix V .

III. Horizontal

The horizontal matrix H is much simpler to calculate, though it takes much longer to compute in practice. For N nodes, we initialize the zero $N \times N$ matrix and iterate through every distinct pair of nodes. If the i th and j th node can horizontally diffuse to each other, then we calculate their distance d according to our exact distance function we specified earlier. Then, we input this into the kernel $k(d) = \exp(-\alpha \cdot d^2)$, and we set entries H_{ij} and H_{ji} equal to that kernel.

Last, we get the row sums of H , all of which are nonzero, and divide each row by its row sum to normalize H and make it a row-stochastic matrix.

IV. Diffusion

Lastly, to compute our final diffusion operator T whose eigenvalues and eigenvectors we will compute, we make a convex combination of our vertical and horizontal diffusion operators:

$$T := \omega V + (1 - \omega)H.$$

Note that because V and H are row-stochastic, the convex combination of them is also row-stochastic.